

PROJECT SYNOPSIS OF MAJOR PROJECT

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

API-BASED RAG KNOWLEDGE CHATBOT: A FULL-STACK RETRIEVAL-AUGMENTED GENERATION PLATFORM FOR API AND PDF DATA SOURCES

Submitted by: Abdur Ursul Khan

University Roll No.: _____

Class Roll No.: _____

Branch: CSE

Batch/Year: _____

Faculty Guide: _____

TABLE OF CONTENTS

1. Introduction
2. Rationale
3. Objectives
4. Literature Review
5. Feasibility Study
6. Methodology / Planning
7. Facilities Required
8. Expected Outcomes
9. References

1. INTRODUCTION

The API-Based RAG Knowledge Chatbot is a full-stack web application developed to transform raw external information sources into an intelligent conversational system. The project allows authenticated users to connect structured API endpoints or upload PDF documents, ingest the data into a vector database, and interact with that indexed knowledge through source-specific chat sessions. The system combines retrieval-augmented generation (RAG), semantic search, and large language model reasoning to produce grounded answers from user-owned data.

The backend is implemented using Django and Django REST Framework, while the frontend is built with Next.js, React, TypeScript, and Tailwind CSS. The application supports email/password authentication, Google OAuth login, secure cookie-based JWT session handling, document ingestion, semantic embeddings, Qdrant vector storage, and conversational querying through a Groq-hosted large language model. The platform is designed as a practical AI knowledge workspace for business, technical, and document-centric use cases.

2. RATIONALE

Modern organizations and developers work with fragmented information spread across REST APIs, dashboards, and PDF files. Traditional chatbots often answer from general pretrained knowledge and may fail to provide domain-specific, source-grounded responses. Similarly, raw APIs and uploaded documents are not directly usable by non-technical users in a conversational workflow.

This project addresses that gap by creating a unified RAG platform where users can register their own sources, ingest them into a searchable semantic index, and ask contextual questions through a controlled chat interface. The system improves accessibility of enterprise or personal knowledge, reduces manual searching effort, and provides a foundation for accurate, source-aware AI assistance. Its value lies in combining source management, retrieval, authentication, and conversational AI in a single deployable product.

3. OBJECTIVES

1. To develop a full-stack RAG application that accepts both API-based and PDF-based data sources.
2. To implement secure user authentication using email/password login, Google OAuth, and cookie-based JWT handling.
3. To design a source ingestion pipeline that normalizes incoming data, generates embeddings, and stores vectors in Qdrant.
4. To enable source-specific chat sessions where responses are generated only from retrieved context.
5. To support customizable AI agent roles so that answers can be tailored for business, technical, or compliance-oriented use cases.
6. To provide an intuitive frontend for source management, dashboard monitoring, and chat interaction.

4. LITERATURE REVIEW

Recent developments in retrieval-augmented generation show that combining dense vector retrieval with large language models improves factual grounding compared to standalone generative systems. Sentence-transformer based embeddings are commonly used for semantic similarity tasks because they provide efficient low-dimensional representations suitable for production systems. Vector databases such as Qdrant have also gained importance for storing and searching embeddings at low latency.

Research and industry practice in modern web engineering indicate that full-stack frameworks benefit from clear separation between authentication, data ingestion, indexing, and user interaction layers. React and Next.js are widely adopted for responsive user interfaces, while Django REST Framework remains a strong choice for secure API-driven backends. The present project builds on these ideas by integrating retrieval, session-aware chat, and multi-source indexing into a unified academic prototype.

5. FEASIBILITY STUDY

Technical Feasibility:

The project uses established technologies including Django, Django REST Framework, Next.js, React, TypeScript, Tailwind CSS, SentenceTransformers, Qdrant, and Groq-compatible LLM APIs. PDF extraction is handled through `pypdf`, while structured API data is fetched and normalized automatically. The architecture is modular and practical for local development as well as cloud deployment.

Economic Feasibility:

Most core tools used in the project are open-source. SQLite can support development and small-scale deployment, while Qdrant offers self-hosted and managed options. The frontend and backend can be deployed using low-cost or student-friendly hosting platforms. Hence, the overall implementation cost is manageable.

Operational Feasibility:

The system is easy to operate for end users because the workflow is straightforward: authenticate, add a source, trigger ingestion, and begin chatting. The dashboard and source management interface reduce operational complexity. The application can be adapted for study material analysis, business reporting, documentation support, or internal knowledge access.

6. METHODOLOGY / PLANNING

The project follows a modular development methodology with clear subsystem separation:

Phase 1: System Design and Authentication Layer

The backend project structure was created in Django with dedicated apps for accounts, sources, and chat. Authentication was implemented using JWT-based cookies, email/password registration, login APIs, refresh token logic, and Google OAuth integration.

Phase 2: Source Management Module

Database models were designed to store API and PDF source metadata, ingestion status, document counts, synchronization timestamps, and optional AI agent instructions. REST endpoints were created for listing, creating, deleting, ingesting, and re-syncing sources.

Phase 3: Retrieval and Indexing Pipeline

API responses are fetched through HTTP requests and normalized into textual records. PDF files are parsed page-wise and chunked into overlapping segments. Sentence embeddings are generated using the `all-MiniLM-L6-v2` model and stored in Qdrant collections unique to each user source.

Phase 4: Conversational Query Module

For each source, users can create or reuse chat sessions. User questions are embedded and matched against indexed vectors through semantic search. The retrieved context is then passed to a large language model with explicit instructions to answer only from provided data.

Phase 5: Frontend Interface and Integration

The Next.js frontend provides a landing page, login interface, dashboard, source creation page, and per-source chat screen. The frontend communicates with the backend using a reusable API utility that also handles token refresh on authentication failure.

Phase 6: Testing and Deployment Readiness

The complete workflow was validated across source creation, ingestion, retrieval, and response generation. The design supports future migration to PostgreSQL, background task queues, and production-grade storage for larger deployments.

7. FACILITIES REQUIRED

Hardware:

1. Computer system with at least 8 GB RAM
2. Multi-core processor
3. Stable internet connection

Software / Tools:

1. Python 3.x
2. Django and Django REST Framework
3. Node.js and npm
4. Next.js, React, and TypeScript
5. Tailwind CSS
6. SQLite database
7. Qdrant vector database
8. SentenceTransformers embedding model
9. Groq API or compatible LLM endpoint
10. `pypdf` for PDF text extraction
11. Code editor such as VS Code

8. EXPECTED OUTCOMES

The project is expected to deliver a working RAG platform capable of converting external data into a conversational knowledge system. Users will be able to upload PDF files or connect APIs, ingest them into a semantic vector store, and receive grounded answers through an intuitive chat interface.

The expected academic and practical outcomes include:

1. Demonstration of a complete retrieval-augmented generation workflow.
2. Integration of full-stack web development with applied artificial intelligence.
3. Support for multi-source document intelligence in a real user interface.
4. Improved accessibility of structured and unstructured data through conversational querying.
5. A strong foundation for future enhancements such as background jobs, multi-tenant deployments, advanced analytics, and production-grade monitoring.

9. REFERENCES

1. Django Documentation. <https://docs.djangoproject.com/>
2. Django REST Framework Documentation. <https://www.django-rest-framework.org/>
3. Next.js Documentation. <https://nextjs.org/docs>
4. React Documentation. <https://react.dev/>
5. Qdrant Documentation. <https://qdrant.tech/documentation/>
6. Sentence Transformers Documentation. <https://www.sbert.net/>
7. PyPDF Documentation. <https://pypdf.readthedocs.io/>
8. Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
9. Groq API / OpenAI-compatible chat completion documentation.